Digital Burnbag: Cryptographic Vaults with Provable Destruction and Non-Access Verification for Decentralized Secure Storage

Jessica Mulein
Digital Defiance
jessica@digitaldefiance.org

Abstract. We present Digital Burnbag, a decentralized secure storage platform that provides two cryptographic guarantees absent from existing cloud storage and whistleblower protection systems: *provable destruction* — a publicly verifiable proof that encryption keys and file-reconstruction metadata have been irrecoverably destroyed — and *proof of non-access* — cryptographic evidence that stored secrets were never read. The system combines a three-layer vault protection model (SHA3-512 Merkle commitment seals, blockchain ledger audit, and custodial ECIES double-encryption) with an automated canary protocol engine that monitors user activity across 40+ third-party services and executes programmable responses — including cryptographic destruction, targeted distribution to journalists or attorneys, and public disclosure — when configurable conditions are met. Files are organized into *Vault Containers* — named, independently governed collections that provide container-level access control, canary bindings, quorum policies, aggregate non-access verification, and atomic cascade destruction, with a seal-break confirmation protocol that warns users before irreversibly mutating a file's cryptographic seal. Built on the BrightChain Owner-Free Filesystem, where all data is stored as XOR-whitened TUPLE blocks providing plausible deniability at the storage layer, Digital Burnbag adds file management, composable POSIX-style access control, multi-recipient key wrapping, quorum-governed sensitive operations, and three-tier external sharing with end-to-end encryption. We formalize the vault lifecycle protocol, analyze the security properties of the commitment scheme and seal mechanism, describe the canary protocol execution model with cascading triggers and duress detection, and present 28 machine-verifiable correctness properties validated through property-based testing and end-to-end integration tests using live cryptographic implementations with zero mocked components. The system is implemented as a production TypeScript library with browser compatibility, deterministic binary serialization, and a repository-pattern architecture enabling deployment across Node.js and browser environments.

Keywords: provable destruction, non-access verification, cryptographic erasure, dead man's switch, canary protocol, Merkle commitment tree, ECIES, Owner-Free Filesystem, decentralized storage, whistleblower protection, access seal, blockchain audit, vault container

---

I. Introduction

The problem of secure data disposal in distributed systems remains fundamentally unsolved. Cloud storage providers offer "delete" operations that remove pointers to data without guaranteeing erasure of the underlying bytes [1]. Encrypted messaging applications provide forward secrecy for communications in transit but offer no mechanism to prove that stored data has been destroyed [2]. Whistleblower platforms such as SecureDrop [3] protect source anonymity during submission but provide no cryptographic guarantee that submitted materials cannot be silently accessed by

platform operators, nor any mechanism for automated release if the source becomes incapacitated.

Digital Burnbag addresses these gaps with a system designed for the most adversarial threat models: journalists protecting sources under government surveillance, whistleblowers requiring guaranteed data expiration, activists operating in hostile jurisdictions, legal professionals managing privileged materials, and individuals planning digital estate succession. The system's name derives from the physical "burn bag" — a container used by intelligence agencies to collect classified documents for high-temperature destruction — but the digital variant provides something its physical counterpart never could: *mathematical proof* that the destruction occurred and that the contents were never read.

The core insight is that provable destruction and non-access verification are not merely features to be bolted onto existing storage systems — they require a fundamentally different architecture. In Digital Burnbag, every file is protected by a *Vault* that encapsulates the file's encryption key and reconstruction recipe inside a cryptographic commitment scheme. The Vault's three-layer protection model ensures that:

1. Any access to the Vault's contents irreversibly mutates a cryptographic seal, providing a fast local indicator of tampering.
2. Every Vault operation is recorded on an append-only blockchain ledger *before* execution, providing tamper-proof ground truth that survives snapshot-restore attacks.
3. The Vault's master secret is double-encrypted under a Custodian's ECIES public key, preventing raw-byte access without a governed, ledger-recorded key release.

   When a Vault is destroyed, the system reveals the master secret (the Merkle tree seed) in a signed destruction proof. Any third party can independently verify this proof by recomputing the Merkle tree and checking the signature — no trust in the platform operator is required. When a Vault has never been accessed, the pristine cryptographic seal combined with the absence of read entries on the ledger constitutes a proof of non-access that is independently verifiable.

   Beyond the Vault primitive, Digital Burnbag provides a complete file management platform with automated *canary protocols* — programmable rules that monitor user activity across dozens of third-party services (fitness trackers, social media, developer platforms, IoT devices) and execute configurable responses when conditions are met. A journalist can configure a dead man's switch that distributes encrypted documents to predetermined recipients if the journalist fails to check in for 72 hours. An activist can set a duress code that, when entered instead of a normal password, immediately triggers cryptographic destruction of sensitive materials while appearing to function normally.

   This paper makes the following contributions:

4. **Three-Layer Vault Protection Model.** We formalize a vault architecture combining SHA3-512 Merkle commitment seals, blockchain ledger audit, and custodial ECIES double-encryption to provide provable destruction and non-access verification (Section III).
5. **Cryptographic Seal Mechanism.** We describe an HMAC-SHA3-512 domain-separated seal that irreversibly mutates on access, providing O(1) local tamper detection with 2^256 brute-force resistance (Section IV).
6. **Formal Algorithms.** We present 11 algorithms in pseudocode — covering tree construction, seal derivation, vault creation/read/destruction, proof verification, non-access verification, memory erasure, Bloom witnesses, and binary serialization — each mapped to production implementations and validated by property-based tests (Section VI).
7. **Vault Container Architecture.** We introduce Vault Containers as the top-level organizational unit, providing container-level ACLs, canary bindings, quorum governance, aggregate non-access verification, atomic cascade destruction, and a seal-break confirmation protocol that warns users before irreversibly breaking a file's cryptographic seal (Section VIII).
8. **Canary Protocol Engine.** We present a programmable automation system supporting three trigger conditions (presence, absence, duress), 16 protocol actions, cascading triggers with configurable delays, and integration with 40+ third-party activity providers (Section VII).
9. **Multi-Recipient Key Wrapping.** We describe a key wrapping table architecture that enables multi-recipient encrypted sharing without re-encrypting file content, supporting internal members, ephemeral share links with URL-fragment key transport, and recipient-provided public keys (Section VIII).

10. **28 Machine-Verifiable Correctness Properties.** We present a comprehensive set of formal properties covering vault lifecycle, seal integrity, destruction proof verification, ledger consistency, custodial security, and adversarial attack resistance, validated through property-based testing and end-to-end integration tests using live cryptographic implementations (Section X).

The remainder of this paper is organized as follows. Section II surveys related work and provides a comparative summary. Section III describes the vault architecture and three-layer protection model. Section IV details the Merkle commitment tree and access seal mechanism. Section V formalizes the vault lifecycle protocol. Section VI presents the core algorithms in pseudocode. Section VII presents the canary protocol engine. Section VIII describes the file platform and sharing architecture, including a scenario walkthrough. Section IX analyzes performance and storage costs. Section X analyzes security properties. Section XI presents the correctness properties and testing methodology. Section XII describes the implementation. Section XIII discusses ethical considerations and dual-use analysis. Section XIV discusses limitations and future work. Section XV concludes.

II. Related Work

A. Secure Deletion and Cryptographic Erasure

The challenge of secure deletion in storage systems has been studied extensively. Reardon et al. [4] surveyed secure deletion techniques and identified cryptographic erasure — destroying encryption keys rather than overwriting data — as the most practical approach for distributed systems. The NIST Guidelines for Media Sanitization (SP 800-88) [5] acknowledge that cryptographic erasure is an acceptable sanitization method when the encryption is sufficiently strong, but provide no mechanism for *proving* that erasure occurred.

Vanish [6] proposed self-destructing data using distributed hash table key storage with natural churn-based expiration, but was shown to be vulnerable to Sybil attacks that could recover keys before expiration [7]. Ephemerizer [8] introduced a trusted third party that deletes keys on a schedule, but requires trust in the third party and provides no non-access verification. Digital Burnbag differs from these approaches in three ways: (1) destruction produces a publicly verifiable cryptographic proof rather than relying on trust assumptions, (2) the seal mechanism provides evidence of non-access that is independent of the destruction proof, and (3) the custodial double-encryption layer prevents bypass even with raw storage access.

B. Whistleblower and Source Protection Platforms

SecureDrop [3], developed by the Freedom of the Press Foundation, provides anonymous document submission via Tor hidden services. GlobaLeaks [9] offers a similar capability with a focus on regulatory compliance. Both systems protect source anonymity during submission but provide no mechanism for automated release, provable destruction, or non-access verification after submission. The platform operator can silently access submitted documents without detection.

WikiLeaks pioneered large-scale document disclosure but operates as a centralized editorial organization with full access to submitted materials [10]. The "insurance file" mechanism — publishing encrypted archives with the threat of key release — provides a crude dead man's switch but offers no granularity, no automated triggers, and no proof that the unencrypted contents have not been accessed.

Digital Burnbag's canary protocol engine provides the automated release capability that these platforms lack, while the vault's non-access verification ensures that even the platform operator cannot silently read stored documents.

C. Dead Man's Switches and Canary Systems

The concept of a "warrant canary" — a regularly published statement that no secret government subpoenas have been received, whose absence signals that one has — has been adopted by organizations including Apple, Reddit, and Cloudflare. The EFF and partners launched Canary Watch [11] to track these statements systematically. However, traditional warrant canaries are binary (present or absent), manually maintained, and provide no automated response capability.

DeadManSwitch.net and similar services offer email-based dead man's switches but rely on centralized servers, provide no cryptographic guarantees, and support only email notification as a response action. Digital Burnbag's canary protocol engine extends the dead man's switch concept with: (1) multi-provider activity monitoring across 40+ services for robust proof-of-life detection, (2) programmable response actions including cryptographic destruction, targeted distribution, and public disclosure, (3) cascading triggers with configurable delays enabling graduated responses, and (4) duress detection that triggers emergency protocols when a special code is entered under coercion.

D. Merkle Trees and Commitment Schemes

Merkle trees [12] provide efficient membership proofs and have been widely adopted in blockchain systems [13], certificate transparency [14], and verifiable data structures [15]. Digital Burnbag uses a SHA3-512 Merkle tree not for membership proof in the traditional sense, but as a *commitment scheme* — the tree root commits to a secret seed, and revealing the seed in the destruction proof allows any verifier to recompute the tree and verify the commitment. The Bloom filter witness provides O(1) probabilistic membership checks as a complement to the O(log N) deterministic Merkle proofs.

E. Owner-Free Filesystems and Plausible Deniability

The Owner-Free Filesystem (OFF) [16] introduced XOR-based block whitening for plausible deniability in distributed storage. BrightChain [17] extended the OFF model with ECIES-based encrypted block types, hierarchical Constituent Block Lists, and storage pool isolation. Digital Burnbag builds on BrightChain's storage layer, inheriting plausible deniability at the block level while adding file-level access control, provable destruction, and automated canary protocols at the application layer.

F. Comparative Summary

Table I summarizes the capabilities of Digital Burnbag against existing systems across the dimensions most relevant to high-stakes secure storage.

Capability	SecureDrop	GlobaLeaks	WikiLeaks Insurance	Vanish	Ephemerizer	OneDrive/iCloud	Digital Burnbag
Provable destruction	No	No	No	No	No	No	**Yes**
Non-access verification	No	No	No	No	No	No	**Yes**
Automated dead man's switch	No	No	Crude (manual)	TTL-based	TTL-based	No	**Yes (40+ providers)**
Duress detection	No	No	No	No	No	No	**Yes**
Plausible deniability (storage)	No	No	No	No	No	No	**Yes (XOR whitening)**
Multi-recipient E2E encryption	No	No	No	No	No	No	**Yes (key wrapping)**
Quorum-governed operations	No	No	No	No	No	No	**Yes**
Cascading trigger responses	No	No	No	No	No	No	**Yes**
Source anonymity	Yes (Tor)	Yes (Tor)	Partial	N/A	N/A	No	**Yes (OFF + Tor)**
Public verifiability of proofs	No	No	No	No	No	No	**Yes**
Decentralized (no single operator)	No	No	No	Partial (DHT)	No	No	**Yes**
Browser-compatible client	Yes	Yes	N/A	No	No	Yes	**Yes**

III. Vault Architecture

A. Overview

The Digital Burnbag Vault is the central cryptographic primitive of the system. A Vault encapsulates a *Secret Payload* — an AES-256-GCM encrypted bundle containing a file's encryption key and its *Recipe* (the ordered list of block identifiers needed to reconstruct the file from the BrightChain block store). The Vault is protected by a three-layer model designed to ensure that any access leaves an indelible, externally verifiable trace.

Layer 1: Cryptographic Seals (Access Seal). A SHA3-512 Merkle tree commitment scheme provides fast, local verification that a Vault's internal state has not been mutated by a read operation. The Vault's master secret — a 32-byte *Tree Seed* — is used to derive a Merkle tree of depth D (default 10, yielding 1,024 leaves). The tree root serves as a public commitment to the seed. An HMAC-SHA3-512 *Access Seal* derived from the seed with a domain separator ("burn-bag-v1-pristine" or "burn-bag-v1-accessed") provides O(1) verification of whether the Vault has ever been read. Any read operation irreversibly mutates the seal from the pristine domain to the accessed domain.

Layer 2: Blockchain Ledger Audit. Every Vault operation — creation, read request, key release, and destruction — is recorded as a signed, hash-chained entry on an append-only blockchain Ledger *before* the operation executes. The ledger is the ground truth: an attacker who snapshots and restores Vault bytes cannot erase the ledger record of a read without breaking the hash chain. The `LedgerVerifier` component cross-checks the seal state against ledger records to detect snapshot-restore tampering — if the seal is pristine but the ledger contains read entries, a `SealLedgerInconsistencyError` is raised.

Layer 3: Custodial Double-Encryption. The Vault's Tree Seed is additionally encrypted under a Custodian's ECIES public key (secp256k1). Reading the Vault requires a key release from the Custodian, and that key release is itself recorded on the ledger. The default `LedgerQuorumCustodian` implementation requires a configurable quorum of admin signers to co-sign the release, preventing unilateral access. Even with raw access to Vault storage bytes, an attacker cannot decrypt without a visible, auditable key release through the governed ledger.

B. Vault Internals

The Vault's internal state comprises:

- `encryptedPayload`: AES-256-GCM ciphertext of the Secret Payload
- `iv`, `authTag`: AES-GCM initialization vector (12 bytes) and authentication tag (16 bytes)
- `pbkdf2Salt`: Salt used for PBKDF2 key derivation from the Tree Seed
- `encryptedTreeSeed`: The Tree Seed encrypted under the Custodian's ECIES public key
- `custodialPublicKey`: 33-byte compressed secp256k1 public key identifying the Custodian
- `creationLedgerEntryHash`: SHA3-512 hash of the `vault_created` ledger entry
- `merkleRoot`: 64-byte SHA3-512 Merkle tree root
- `treeDepth`: Depth of the commitment tree (minimum 8)
- `accessSeal`: 64-byte HMAC-SHA3-512 seal (mutable)
- `state`: One of `Sealed`, `Accessed`, or `Destroyed`

  The Vault exposes a single public operation: `read(creatorPrivateKey)`, which returns the decrypted encryption key and Recipe. The state machine enforces irreversible transitions: `Sealed → Accessed` on first read, `Sealed → Destroyed` or `Accessed → Destroyed` on destruction, and all operations rejected in the `Destroyed` state.

C. Verification Bundle

A *Verification Bundle* is the public commitment data that enables third-party verification of destruction proofs and non-access claims. It contains:

- `merkleRoot`: The SHA3-512 root of the commitment tree
- `accessSeal`: The current HMAC-SHA3-512 seal value
- `creatorPublicKey`: 33-byte compressed secp256k1 public key of the Vault creator
- `bloomWitness`: Serialized Bloom filter over all tree nodes

- `treeDepth`: Depth of the commitment tree
- `destructionProof`: (Optional) The signed destruction proof, if the Vault has been destroyed

  The bundle is serialized to a deterministic binary format with CRC-16 integrity checking, enabling transport and storage without JSON ambiguity.

IV. Merkle Commitment Tree and Access Seal

A. Tree Construction

The Merkle Commitment Tree is constructed from a 32-byte Tree Seed as follows:

Leaf derivation. For a tree of depth D, 2^D leaves are derived:

$$\text{leaf}[i] = \text{SHA3-512}(\text{treeSeed} \mid \text{bigEndian32}(i)) \quad \text{for } i \in [0, 2^D)$$

Internal nodes. Each internal node is the hash of its children:

$$\text{node} = \text{SHA3-512}(\text{leftChild} \mid \text{rightChild})$$

Root. The Merkle root is the single node at the top of the tree.

The minimum depth is 8 (256 leaves), providing a commitment space of 511 total nodes. The default depth is 10 (1,024 leaves, 2,047 total nodes). Depths above 16 are permitted but carry significant resource costs (~8 MB at depth 16 with 64-byte hashes).

B. Merkle Proofs for Selective Disclosure

For any leaf at index i, a Merkle proof consists of D sibling hashes from leaf to root. The proof size is O(D) = O(log N), enabling efficient selective disclosure of individual commitment tree leaves without revealing the Tree Seed. Verification recomputes hashes from the leaf to the root using the sibling path and compares against the published Merkle root.

This capability enables advanced use cases such as batch burn bags (multiple secrets per Vault) where individual secrets can be selectively disclosed via Merkle proofs while the remaining secrets stay committed but unrevealed.

C. Access Seal Derivation

The Access Seal is an HMAC-SHA3-512 value derived from the Tree Seed with a domain separator:

$$\text{seal} = \text{HMAC-SHA3-512}(\text{key}=\text{treeSeed}, \text{data}=\text{UTF8}(\text{domain}))$$

Two domains are defined:

- **Pristine:** `"burn-bag-v1-pristine"` — the seal value for a Vault that has never been read
- **Accessed:** `"burn-bag-v1-accessed"` — the seal value after the first read operation

  The domain-separated HMAC construction prevents length-extension attacks and ensures that the pristine and accessed seals are cryptographically independent — knowledge of one reveals nothing about the other without the Tree Seed.

Seal verification is O(1): given the Tree Seed, compute the HMAC with the pristine domain and compare against the stored seal. If they match, the Vault has never been read. If the accessed domain matches instead, the Vault has been read. If neither matches, the seal has been tampered with.

Brute-force resistance. The Tree Seed is 256 bits of entropy from `crypto.getRandomValues()`. An attacker attempting to forge a pristine seal must find a 32-byte value X such that HMAC-SHA3-512(X, "burn-bag-v1-pristine") equals the stored seal — a preimage attack on HMAC-SHA3-512, which requires 2^256 operations.

D. Bloom Filter Witness

A Bloom filter is constructed over all nodes in the commitment tree (leaves and internal nodes), providing O(1) probabilistic membership checks. The Bloom witness enables fast verification that a candidate node belongs to the commitment tree without requiring the full tree structure. The false positive rate is configurable (default 0.001).

The Bloom witness complements Merkle proofs: Bloom queries are O(1) but probabilistic, while Merkle proofs are O(log N) but deterministic. Together they provide a flexible verification toolkit.

V. Vault Lifecycle Protocol

A. Vault Creation

The `VaultFactory.create()` protocol proceeds as follows:

1. Generate 32-byte random Tree Seed via `crypto.getRandomValues()`.
2. Derive 2^D leaves and build the Merkle commitment tree.
3. Derive the pristine Access Seal: HMAC-SHA3-512(treeSeed, "burn-bag-v1-pristine").
4. Derive an AES-256 key from the Tree Seed via PBKDF2 with a random salt.
5. Serialize the Recipe via deterministic binary serialization with CRC-16.
6. Concatenate the encryption key and serialized Recipe with a 4-byte length prefix.
7. Encrypt the concatenated payload via AES-256-GCM.
8. Encrypt the Tree Seed under the Custodian's ECIES public key.
9. **Record a `vault_created` entry on the blockchain ledger.** If the ledger write fails, abort — no Vault is created.
10. Build the Bloom witness over all tree nodes.
11. Assemble the Vault and Verification Bundle.
12. Erase the plaintext Tree Seed and derived AES key from memory (overwrite with random bytes, then zeros).

    The ledger-first design (step 9 before step 11) ensures that no Vault exists without a corresponding ledger record.

B. Vault Read

The `Vault.read()` protocol:

1. Assert state is not `Destroyed`.
2. **Record a `vault_read_requested` entry on the ledger.** If the write fails, abort — seal unchanged.

3. **Request custodial key release.** The Custodian verifies the requester's authorization, checks admin quorum signatures, records a `key_released` entry on the ledger, and returns the decrypted Tree Seed. If the Custodian refuses, abort — seal unchanged.
4. Derive the AES key from the Tree Seed via PBKDF2.
5. Decrypt the payload via AES-256-GCM.
6. If state is `Sealed`, mutate the Access Seal to the accessed domain. Transition to `Accessed`.
7. Parse the decrypted payload: extract the encryption key and deserialize the Recipe.
8. Erase the derived key and plaintext Tree Seed from memory.
9. Return the encryption key and Recipe.

   The critical ordering — ledger write (step 2) and custodial key release (step 3) *before* seal mutation (step 6) — ensures that the ledger always reflects reality. If the ledger write or custodial release fails, the seal remains pristine and the Vault state is unchanged.

C. Vault Destruction

The `DestructionEngine.destroy()` protocol:

1. Assert state is not `Destroyed`.
2. **Record a `vault_destroyed` entry on the ledger.** If the write fails, abort.
3. Obtain the Tree Seed (from prior read or custodial release for destruction).
4. Generate a 32-byte random nonce.
5. Record the current UTC timestamp.
6. Construct the message: `treeSeed || nonce || bigEndian64(timestamp)`.
7. Sign the message with the creator's secp256k1 private key.
8. Erase the encrypted payload, encrypted Tree Seed, and plaintext Tree Seed from memory (overwrite with random, then zeros).
9. Mark the Vault as `Destroyed`.
10. Return the Destruction Proof containing the revealed Tree Seed, nonce, timestamp, and signature.

    The revealed Tree Seed is the key to verification: any third party can recompute the Merkle tree from the seed, verify that the root matches the published Merkle root in the Verification Bundle, verify the signature, and check the seal status to determine whether the Vault was ever read before destruction.

D. Destruction Proof Verification

The `ProofVerifier.verify()` protocol:

1. Reconstruct the signed message from the proof fields.
2. Verify the secp256k1 signature against the creator's public key from the Verification Bundle.
3. Verify the Merkle tree: recompute the root from the revealed Tree Seed and compare against the bundle's Merkle root.
4. Validate the timestamp (within configurable tolerance, default 300 seconds).
5. Check seal status: compute the pristine seal from the revealed Tree Seed and compare against the bundle's Access Seal to determine whether the Vault was ever read.

6. Return a composite result: signature validity, chain validity, timestamp validity, and seal status (`pristine`, `accessed`, or `unknown`).

E. Non-Access Verification

The `LedgerVerifier.verifyNonAccess()` protocol provides the strongest non-access guarantee by cross-checking two independent sources:

1. **Seal check (local, fast):** Verify that the Access Seal matches the pristine domain.
2. **Ledger walk (distributed, authoritative):** Walk all ledger entries referencing the Vault's creation hash. Count entries of type `vault_read_requested` and `key_released`.
3. **Cross-check:** If the seal is pristine and the ledger contains zero read/key-release entries, non-access is confirmed. If the seal is pristine but the ledger contains read entries (indicating a snapshot-restore attack), raise a `SealLedgerInconsistencyError`.

   This dual-source verification defeats the most sophisticated attack: an adversary who reads the Vault, copies the data, and then restores the Vault bytes to their pre-read state. The seal would appear pristine, but the ledger — which is append-only and hash-chained — retains the record of the read.

VI. Algorithms

This section presents the core algorithms of the Digital Burnbag vault system in pseudocode. Each algorithm corresponds directly to a production TypeScript implementation and is validated by the correctness properties in Section X.

Algorithm 1: Merkle Commitment Tree Construction

```
FUNCTION BuildMerkleTree(treeSeed: bytes[32], D: int) → MerkleTree
  REQUIRE D ≥ 8
  N ← 2^D

  // Derive leaves
  FOR i ← 0 TO N-1 DO
    leaf[i] ← SHA3-512(treeSeed ‖ BigEndian32(i))

  // Build tree bottom-up
  levels[0] ← leaf[0..N-1]
  FOR d ← 0 TO D-1 DO
    FOR j ← 0 TO |levels[d]|/2 - 1 DO
      levels[d+1][j] ← SHA3-512(levels[d][2j] ‖ levels[d][2j+1])

  root ← levels[D][0]
  RETURN MerkleTree{root, leaves, depth: D, levels}
```

Validated by: Property 2 (round-trip), Property 3 (wrong seed rejection), Property 28 (collision resistance). Implementation: `MerkleCommitmentTree.build()` in `crypto/merkle-commitment-tree.ts`.

Algorithm 2: Merkle Proof Generation and Verification

```
FUNCTION GenerateProof(tree: MerkleTree, leafIndex: int) → MerkleProof
  REQUIRE 0 ≤ leafIndex < 2^tree.depth
  siblings ← []
  directions ← []
  idx ← leafIndex
  current ← tree.leaves

  FOR d ← 0 TO tree.depth − 1 DO
    siblingIdx ← idx XOR 1          // flip least significant bit
    siblings.append(current[siblingIdx])
    directions.append((idx AND 1) = 0)  // true = sibling is right
    // Recompute next level
    next ← []
    FOR j ← 0 TO |current|/2 − 1 DO
      next[j] ← SHA3-512(current[2j] ‖ current[2j+1])
    current ← next
    idx ← idx >> 1

  RETURN MerkleProof{leafValue: tree.leaves[leafIndex],
                     leafIndex, siblings, directions}

FUNCTION VerifyProof(proof: MerkleProof, expectedRoot: bytes[64]) → bool
  hash ← proof.leafValue
  FOR i ← 0 TO |proof.siblings| − 1 DO
    IF proof.directions[i] THEN     // sibling is on the right
      hash ← SHA3-512(hash ‖ proof.siblings[i])
    ELSE                             // sibling is on the left
      hash ← SHA3-512(proof.siblings[i] ‖ hash)
  RETURN ConstantTimeEqual(hash, expectedRoot)
```

Validated by: Property 20 (selective disclosure round-trip). Implementation: `MerkleCommitmentTree.generateProof()` *and* `verifyProof()` *in* `crypto/merkle-commitment-tree.ts`*.*

Algorithm 3: Access Seal Derivation and Verification

```
CONSTANT PRISTINE_DOMAIN ← "burn-bag-v1-pristine"
CONSTANT ACCESSED_DOMAIN ← "burn-bag-v1-accessed"

FUNCTION DeriveSeal(treeSeed: bytes[32], domain: string) → bytes[64]
  RETURN HMAC-SHA3-512(key = treeSeed, data = UTF8(domain))

FUNCTION VerifyPristine(treeSeed: bytes[32], seal: bytes[64]) → bool
  expected ← DeriveSeal(treeSeed, PRISTINE_DOMAIN)
  RETURN ConstantTimeEqual(expected, seal)

FUNCTION VerifyAccessed(treeSeed: bytes[32], seal: bytes[64]) → bool
  expected ← DeriveSeal(treeSeed, ACCESSED_DOMAIN)
  RETURN ConstantTimeEqual(expected, seal)
```

Validated by: Property 4 (seal mutation on read), Property 5 (failed decryption preserves seal), Property 22 (seal forgery infeasibility). Implementation: `AccessSeal` *class in* `crypto/access-seal.ts`*.*

Algorithm 4: Vault Creation

```
FUNCTION CreateVault(encKey: bytes[], recipe: Recipe,
                     creatorPrivKey: bytes[32]) → (Vault, VerificationBundle)
  // Step 1: Generate random tree seed
  treeSeed ← RandomBytes(32)

  TRY
    // Step 2: Build commitment tree
    tree ← BuildMerkleTree(treeSeed, CONFIG.treeDepth)

    // Step 3: Derive pristine seal
    seal ← DeriveSeal(treeSeed, PRISTINE_DOMAIN)

    // Step 4: Derive AES key via PBKDF2
    salt ← RandomBytes(32)
    aesKey ← PBKDF2-SHA256(treeSeed, salt, CONFIG.iterations, 32)

    // Step 5: Serialize and encrypt payload
    serializedRecipe ← RecipeSerializer.serialize(recipe)
    payload ← BigEndian32(|encKey|) ‖ encKey ‖ serializedRecipe
    (ciphertext, iv, tag) ← AES-256-GCM.Encrypt(payload, aesKey)

    // Step 6: Double-encrypt tree seed under custodian
    (encTreeSeed, custPubKey) ← Custodian.EncryptTreeSeed(treeSeed)

    // Step 7: Record creation on ledger (BEFORE assembling vault)
    creatorPubKey ← ECIES.GetPublicKey(creatorPrivKey)
    ledgerHash ← LedgerGateway.RecordCreation(tree.root, creatorPubKey)
    // If ledger write fails → LedgerWriteError, abort

    // Step 8: Build Bloom witness
    bloom ← BloomFilter.Create(tree.AllNodes(), fpRate = 0.001)

    // Step 9: Assemble vault and bundle
    vault ← Vault{ciphertext, iv, tag, salt, encTreeSeed, custPubKey,
                  ledgerHash, tree.root, CONFIG.treeDepth, seal,
                  state: SEALED}
    bundle ← VerificationBundle{tree.root, seal, creatorPubKey,
                                bloom.Serialize(), CONFIG.treeDepth}
    RETURN (vault, bundle)
  FINALLY
    // Step 10: Erase sensitive material
    MemoryEraser.Wipe(treeSeed)
    MemoryEraser.Wipe(aesKey)
```

Validated by: Property 1 (payload round-trip), Property 9 (Bloom completeness), Property 15 (ledger records creation), Property 16 (ledger failure aborts), Property 17 (custodial encryption). Implementation: `VaultFactory.create()` in `crypto/vault-factory.ts`.

Algorithm 5: Vault Read

```
FUNCTION ReadVault(vault: Vault, requesterPubKey: bytes[33],
                   adminSigs?: AdminSignature[]) → (bytes[], Recipe)
  ACQUIRE vault.mutex
```

```
  // Step 1: Check state
  IF vault.state = DESTROYED THEN
    THROW VaultDestroyedError

  // Step 2: Record read on ledger BEFORE any decryption
  LedgerGateway.RecordRead(vault.creationLedgerEntryHash)
  // If ledger write fails → LedgerWriteError, seal unchanged

  // Step 3: Request custodial key release
  treeSeed ← Custodian.RequestKeyRelease(
    vault.creationLedgerEntryHash,
    vault.encryptedTreeSeed,
    requesterPubKey, adminSigs)
  // Custodian records key_released on ledger before returning
  // If refused → CustodialKeyReleaseError, seal unchanged

  TRY
    // Step 4: Derive AES key
    aesKey ← PBKDF2-SHA256(treeSeed, vault.pbkdf2Salt,
                           CONFIG.iterations, 32)

    // Step 5: Decrypt payload
    plaintext ← AES-256-GCM.Decrypt(vault.iv,
                  vault.ciphertext ‖ vault.authTag, aesKey)
    // If decryption fails → DecryptionError, seal unchanged

    // Step 6: Mutate seal on first read
    IF vault.state = SEALED THEN
      vault.accessSeal ← DeriveSeal(treeSeed, ACCESSED_DOMAIN)
      vault.state ← ACCESSED

    // Step 7: Parse payload
    keyLen ← BigEndian32(plaintext[0..3])
    encKey ← plaintext[4 .. 4+keyLen-1]
    recipe ← RecipeSerializer.Deserialize(plaintext[4+keyLen ..])

    RETURN (encKey, recipe)
  FINALLY
    // Step 8: Erase sensitive material
    MemoryEraser.Wipe(aesKey)
    MemoryEraser.Wipe(treeSeed)

  RELEASE vault.mutex
```

Validated by: Property 1 (round-trip), Property 4 (seal mutation), Property 5 (failed decryption preserves seal), Property 6 (destroyed rejection), Property 14 (state machine), Property 18 (custodial refusal preserves state), Property 24 (concurrency safety). Implementation: `Vault.read()` in `crypto/vault.ts`.

Algorithm 6: Vault Destruction and Proof Generation

```
FUNCTION DestroyVault(vault: Vault, creatorPrivKey: bytes[32],
                      requesterPubKey: bytes[33],
                      adminSigs?: AdminSignature[]) → DestructionProof
  // Step 1: Check state
  IF vault.state = DESTROYED THEN
    THROW VaultDestroyedError
```

```
  // Step 2: Record destruction on ledger
  LedgerGateway.RecordDestruction(vault.creationLedgerEntryHash)
  // If ledger write fails → LedgerWriteError, abort

  // Step 3: Obtain tree seed via custodial release
  treeSeed ← Custodian.RequestKeyRelease(
    vault.creationLedgerEntryHash,
    vault.encryptedTreeSeed,
    requesterPubKey, adminSigs)

  TRY
    // Step 4: Generate nonce and timestamp
    nonce ← RandomBytes(32)
    timestamp ← CurrentUTCMilliseconds()

    // Step 5: Construct and sign message
    message ← treeSeed ‖ nonce ‖ BigEndian64(timestamp)
    signature ← ECDSA-secp256k1.Sign(creatorPrivKey, message)
    creatorPubKey ← ECIES.GetPublicKey(creatorPrivKey)

    // Step 6: Erase vault contents
    MemoryEraser.Wipe(vault.encryptedPayload)
    MemoryEraser.Wipe(vault.encryptedTreeSeed)

    // Step 7: Mark destroyed
    vault.state ← DESTROYED

    // Step 8: Return proof (treeSeed is REVEALED — this is the key)
    RETURN DestructionProof{treeSeed: Copy(treeSeed),
                            nonce, timestamp, signature, creatorPubKey}
  FINALLY
    MemoryEraser.Wipe(treeSeed)
```

Validated by: Property 7 (proof verification round-trip), Property 8 (corrupted proof rejection), Property 13 (memory wipe), Property 23 (replay rejection), Property 27 (field-level tamper evidence). Implementation: `DestructionEngine.destroy()` in `crypto/destruction-engine.ts`.

Algorithm 7: Destruction Proof Verification

```
FUNCTION VerifyDestructionProof(proof: DestructionProof,
                                bundle: VerificationBundle,
                                tolerance: int = 300s) → VerificationResult
  // Step 1: Reconstruct signed message
  message ← proof.treeSeed ‖ proof.nonce ‖ BigEndian64(proof.timestamp)

  // Step 2: Verify ECDSA signature
  sigValid ← ECDSA-secp256k1.Verify(
    bundle.creatorPublicKey, message, proof.signature)

  // Step 3: Verify Merkle commitment
  computedRoot ← ComputeMerkleRoot(proof.treeSeed, bundle.treeDepth)
  chainValid ← ConstantTimeEqual(computedRoot, bundle.merkleRoot)

  // Step 4: Validate timestamp
  tsValid ← (proof.timestamp ≤ CurrentUTCMilliseconds() + tolerance×1000)

  // Step 5: Determine seal status
```

```
  IF VerifyPristine(proof.treeSeed, bundle.accessSeal) THEN
    sealStatus ← "pristine"    // vault was NEVER read before destruction
  ELSE IF VerifyAccessed(proof.treeSeed, bundle.accessSeal) THEN
    sealStatus ← "accessed"    // vault was read before destruction
  ELSE
    sealStatus ← "unknown"     // seal was tampered with

  // Step 6: Composite result
  valid ← sigValid AND chainValid AND tsValid
  RETURN VerificationResult{valid, sigValid, chainValid, tsValid, sealStatus}
```

Validated by: Property 7 (round-trip), Property 8 (corruption detection), Property 23 (replay rejection), Property 27 (field-level tamper evidence). Implementation: `ProofVerifier.verify()` in `crypto/proof-verifier.ts`.

Algorithm 8: Non-Access Verification (Ledger Cross-Check)

```
FUNCTION VerifyNonAccess(creationHash: bytes[64],
                         accessSeal: bytes[64],
                         treeSeed: bytes[32]) → LedgerVerificationResult
  // Step 1: Local seal check (O(1))
  isPristine ← VerifyPristine(treeSeed, accessSeal)
  isAccessed ← VerifyAccessed(treeSeed, accessSeal)

  // Step 2: Walk ledger entries (authoritative)
  readCount ← 0
  keyReleaseCount ← 0
  FOR EACH entry IN Ledger DO
    IF entry.type = "vault_read_requested"
       AND entry.payload CONTAINS creationHash THEN
      readCount ← readCount + 1
    IF entry.type = "key_released"
       AND entry.payload CONTAINS creationHash THEN
      keyReleaseCount ← keyReleaseCount + 1

  // Step 3: Cross-check
  hasLedgerAccess ← (readCount > 0) OR (keyReleaseCount > 0)

  IF isPristine AND NOT hasLedgerAccess THEN
    RETURN {nonAccessConfirmed: TRUE, consistent: TRUE}

  IF isPristine AND hasLedgerAccess THEN
    // SNAPSHOT-RESTORE ATTACK DETECTED
    RETURN {nonAccessConfirmed: FALSE, consistent: FALSE,
            error: "SealLedgerInconsistencyError"}

  IF isAccessed THEN
    consistent ← hasLedgerAccess
    RETURN {nonAccessConfirmed: FALSE, consistent}

  RETURN {nonAccessConfirmed: FALSE, sealStatus: "unknown"}
```

Validated by: Property 19 (non-access verification), Property 21 (snapshot-restore detection). Implementation: `LedgerVerifier.verifyNonAccess()` in `ledger/ledger-verifier.ts`.

Algorithm 9: Memory Erasure

```
FUNCTION Wipe(buffer: bytes[])
  IF buffer = NULL OR |buffer| = 0 THEN RETURN
  // Pass 1: Overwrite with random to destroy original pattern
  buffer ← CryptoRandomBytes(|buffer|)
  // Pass 2: Zero-fill to leave clean state
  FOR i ← 0 TO |buffer|-1 DO
    buffer[i] ← 0x00
```

Validated by: Property 13 (memory wipe zeros buffer). Implementation: `MemoryEraser.wipe()` in `crypto/memory-eraser.ts`. Note: JavaScript cannot guarantee erasure due to GC copying; this is best-effort.

Algorithm 10: Bloom Witness Construction and Query

```
FUNCTION CreateBloomWitness(tree: MerkleTree,
                            fpRate: float = 0.001) → BloomWitness
  nodes ← tree.AllNodes()    // all leaves + internal nodes
  bf ← BloomFilter.Create(|nodes|, fpRate)
  FOR EACH node IN nodes DO
    bf.Insert(HexEncode(node))
  RETURN BloomWitness{bf}

FUNCTION QueryBloomWitness(witness: BloomWitness,
                           candidate: bytes[64]) → bool
  RETURN witness.bf.Has(HexEncode(candidate))
```

Validated by: Property 9 (no false negatives), Property 10 (serialization round-trip). Implementation: `BloomWitness` class in `crypto/bloom-witness.ts`.

Algorithm 11: Deterministic Binary Serialization (VerificationBundle)

```
FUNCTION SerializeBundle(bundle: VerificationBundle) → bytes[]
  buf ← []
  buf.Append(0x01)                          // version
  buf.Append(bundle.merkleRoot)             // 64 bytes
  buf.Append(bundle.accessSeal)             // 64 bytes
  buf.Append(bundle.creatorPublicKey)       // 33 bytes
  buf.Append(BigEndian32(bundle.treeDepth))
  buf.Append(BigEndian32(|bundle.bloomWitness|))
  buf.Append(bundle.bloomWitness)
  IF bundle.destructionProof ≠ NULL THEN
    buf.Append(0x01)
    buf.Append(SerializeProof(bundle.destructionProof))
  ELSE
    buf.Append(0x00)
  crc ← CRC-16(buf)
  buf.Append(crc)                           // 2 bytes
  RETURN buf

FUNCTION DeserializeBundle(data: bytes[]) → VerificationBundle
  // Verify CRC-16 over all bytes except last 2
```

```
storedCRC ← data[|data|-2 ..]
computedCRC ← CRC-16(data[0 .. |data|-3])
IF storedCRC ≠ computedCRC THEN
  THROW DeserializationError("CRC-16 mismatch")
// Parse fields at fixed offsets...
RETURN VerificationBundle{...}
```

Validated by: Property 11 (serialization round-trip, CRC corruption detection). Implementation: `BundleSerializer` in `serialization/bundle-serializer.ts`.

VII. Canary Protocol Engine

A. Motivation

The vault primitive provides provable destruction and non-access verification, but these capabilities are only useful if they can be triggered automatically under adversarial conditions. A journalist who is detained cannot manually destroy their sources' documents. A whistleblower who is killed cannot release their evidence. The canary protocol engine bridges this gap by automating vault operations based on observable signals from the physical and digital world.

B. Trigger Model

The canary protocol engine supports three trigger conditions:

Presence. The signal is actively detected — the user is logging into services, posting on social media, recording fitness data, or interacting with IoT devices. Presence triggers execute actions *while* the user is active, enabling real-time responses to specific events.

Absence. The signal is *not* detected for a configurable duration. This is the classic dead man's switch: if the user fails to produce proof-of-life signals across their configured providers for a specified period, the system concludes that the user is incapacitated, detained, or deceased, and executes the bound protocol actions. Absence detection uses configurable thresholds with warning periods before final execution.

Duress. The user explicitly signals that they are under coercion by entering a special duress code instead of their normal password. The system appears to function normally (to avoid alerting the coercer) while silently executing all duress-bound protocol actions — typically immediate cryptographic destruction of the most sensitive materials and distribution of pre-selected documents to trusted recipients.

C. Activity Providers

The system integrates with 40+ third-party services organized into categories:

- **Health & Fitness:** Fitbit, Strava, Apple Health, Google Fit, Garmin, Whoop, Oura — detecting daily activity patterns such as steps, heart rate, and sleep
- **Social Media:** Twitter/X, Facebook, Instagram, LinkedIn, Mastodon, Bluesky, Reddit — detecting posting, liking, and interaction patterns
- **Developer Platforms:** GitHub, GitLab, Bitbucket, Stack Overflow — detecting commits, issues, and code review activity

- **Communication:** Slack, Discord, Microsoft Teams, Telegram, Signal — detecting messaging presence and activity
- **Productivity:** Google (Gmail, Calendar), Microsoft 365, Notion, Todoist — detecting email, calendar, and task activity
- **Financial:** Plaid — detecting banking transaction activity
- **IoT & Smart Home:** Home Assistant, SmartThings — detecting physical presence via motion sensors, door sensors, and device usage
- **Gaming:** Steam, Xbox Live, PlayStation Network — detecting gaming activity
- **Special:** Custom webhooks, native BrightChain activity, manual check-in, email ping, SMS ping

  The provider system is extensible via a configuration-driven adapter pattern. Each provider implements a standard interface for querying recent activity, and the canary engine polls providers at configurable intervals. Multiple providers can be combined for redundancy — a user might configure GitHub, Fitbit, and Slack as proof-of-life sources, requiring absence from *all three* before triggering.

D. Protocol Actions

When a trigger condition is met, the canary engine executes one or more of 16 configurable actions:

- **Destruction:** `DeleteFiles`, `DeleteFolders` — cryptographic destruction of bound files/folders via the DestructionEngine, producing blockchain-recorded destruction proofs
- **Distribution:** `EmailFilesAsAttachments` (with optional PGP encryption per recipient), `EmailFilesAsLinks`, `ReleaseToPublic` (submission to news organizations and public disclosure endpoints), `ReleaseToRestricted`, `ReleaseToPassword`, `ReleaseToSelf`
- **Account:** `DeactivateAccount`, `DeleteAccount`
- **Communication:** `SendSMS`, `CallWebhook`
- **Protocol Management:** `DeactivateProtocol`, `EnableSecondaryProtocols`, `RestrictToNone`

  Each action is bound to specific files and/or folders. Folder bindings are resolved recursively — binding a canary to a folder affects all files within it.

E. Cascading Triggers

Canary bindings support cascading execution with configurable delays. A primary binding executes immediately when triggered, then schedules secondary bindings for delayed execution:

1. **Immediate:** Destroy the most sensitive files (e.g., source identities).
2. **After 24 hours:** Distribute evidence packages to attorneys.
3. **After 72 hours:** Release redacted versions to journalists.
4. **After 7 days:** Submit full archives to public disclosure endpoints.

   Cascading actions can be cancelled during the delay period if the trigger condition resolves (e.g., the user checks in). This graduated response model prevents irreversible actions from executing prematurely while ensuring that prolonged incapacitation eventually triggers full disclosure.

F. Dry-Run Simulation

Every canary binding supports a dry-run mode that simulates protocol execution without performing any actions. The dry run resolves all affected files and folders, identifies recipients, and produces a detailed report — enabling users to verify their canary configurations before they are needed. Dry runs are logged to the audit trail.

VIII. File Platform and Sharing Architecture

A. Vault Containers

A *Vault Container* is the top-level organizational unit in Digital Burnbag. Every folder and file lives inside exactly one Vault Container. Users create named containers to group files by purpose, sensitivity, or governance policy — for example, "Source Protection — Project X," "Estate Planning," or "Daily Files."

Each Vault Container provides:

- **Container-level ACL.** A default access control list inherited by all files and folders within the container unless explicitly overridden at the file or folder level.
- **Container-level canary bindings.** Canary protocols can target an entire container (in addition to individual files and folders), enabling "destroy everything in this vault" or "distribute everything in this vault" as a single binding.
- **Container-level quorum governance.** Sensitive operations on any file within the container can require multi-party approval.
- **Aggregate non-access verification.** A container-level non-access check walks every file vault within the container and confirms that all seals are pristine and all ledger entries are clean. If any single file has been accessed, the container-level non-access claim fails.
- **Aggregate seal status.** A dashboard-level summary reports how many file vaults are sealed, accessed, or destroyed within each container — e.g., "3 of 12 files accessed."
- **Atomic cascade destruction.** Destroying a container cascade-destroys every file vault within it, producing individual per-file destruction proofs plus a container-level ledger entry that ties them together. Destruction is best-effort: individual file failures are collected without aborting the cascade.

  The container lifecycle follows a three-state model: `Active` (normal operation), `Locked` (no modifications, reads still work but break seals), and `Destroyed` (all file vaults destroyed, container is a tombstone with proofs).

  The cryptographic Vault per file version is unchanged — the container is a metadata/organizational concept backed by the database, not a new cryptographic primitive. The provable destruction and non-access guarantees all live at the per-file-version Vault level.

B. Seal-Break Confirmation Protocol

Because reading a file irreversibly mutates its cryptographic seal from pristine to accessed, the system implements a two-step access protocol:

1. **Access status check.** Before reading a file, the client calls `checkAccessStatus(fileId)`, which returns the current vault state and, if the vault is sealed, a warning message: *"This file has never been accessed. Reading it will permanently break the cryptographic seal. Non-access can no longer be proven for this file or its vault container."*
2. **Confirmed read.** The client presents the warning to the user. If the user confirms, the read proceeds with `confirmSealBreak: true`. If the user cancels, the seal remains pristine.

   This protocol prevents accidental seal breaks, which is critical because the mutation is irreversible. The warning is a UX concern — the cryptographic layer mutates the seal regardless — but it ensures users make informed

decisions about provability.

C. File Management

Digital Burnbag provides a complete file management system built on the vault primitive:

- **Chunked upload** with server-side session management, per-chunk checksums, and resume capability for unstable connections
- **Versioning** with one Vault per file version, enabling independent encryption, destruction, and non-access proofs per version
- **Folder hierarchy** within each Vault Container, with ACL inheritance — folder permissions cascade to descendants unless explicitly overridden
- **Soft-delete with trash bin** and configurable retention periods, after which files are cryptographically destroyed
- **Search** by filename, tags, and MIME type with ACL-filtered results
- **Storage quotas** with per-user and optional per-container limits, with automatic recalculation after destruction
- **Folder export** to TCBL (Tarball Constituent Block List) archives for bulk operations

D. Access Control

The ACL system uses composable POSIX-style permission flags: `Read`, `Write`, `Delete`, `Share`, `Admin`, `Preview`, `Comment`, `Download`, and `ManageVersions`. These atomic flags are composed into named Permission Sets, with four built-in levels:

- **Viewer:** Read + Preview + Download
- **Commenter:** Read + Preview + Download + Comment
- **Editor:** Read + Write + Preview + Download + Comment + ManageVersions
- **Owner:** All flags

  Organizations can define custom Permission Sets with arbitrary flag combinations. ACL entries support additional constraints: IP range restrictions (CIDR notation), time-window constraints (e.g., business hours only), expiration dates, download blocking (view-only in browser), and reshare control.

E. Multi-Recipient Key Wrapping

File content is encrypted once with a symmetric key (AES-256-GCM). The symmetric key is then wrapped (encrypted) separately for each authorized party under their public key. The file on disk never changes — only the key wrapping table grows:

- **Owner (Vault):** The symmetric key is encrypted inside the Vault under custodial ECIES
- **Internal Members:** ECIES(member.publicKey, symmetricKey)
- **Ephemeral Share Links:** ECIES(ephemeral.publicKey, symmetricKey), with the ephemeral private key embedded in the URL fragment (never sent to the server)
- **Recipient Public Keys:** ECIES(recipient.publicKey, symmetricKey) or PGP wrapping

  Revoking a share removes that entry from the wrapping table without affecting the file or other wrappings. For file destruction, the DestructionService destroys the Vault (which holds the master symmetric key) AND deletes

all key wrapping entries — since the symmetric key is gone, all wrapped copies become useless and the encrypted blocks in the store are irrecoverable.

F. Three-Tier External Sharing

External sharing supports three encryption modes with increasing security:

Server-Proxied (Tier 1). The server reads the Vault to obtain the symmetric key and re-serves the decrypted content over TLS. This provides OneDrive/iCloud-level usability — recipients need only a URL and optional password — but requires trust in the server during the sharing session.

Ephemeral Key Pair (Tier 2). The server generates an ephemeral ECIES key pair, wraps the symmetric key under the ephemeral public key, and returns the ephemeral private key in the URL fragment (`https://...#ephemeralPrivateKey`). The URL fragment is never sent to the server (per RFC 3986), so the server cannot decrypt the content after link creation. Recipients decrypt locally in the browser.

Recipient Public Key (Tier 3). The symmetric key is wrapped under the recipient's own ECIES or PGP public key. Only the intended recipient can decrypt. This provides the strongest guarantee but requires the recipient to have and manage a key pair.

G. Quorum-Governed Operations

Sensitive operations — destruction, external sharing, bulk deletion, ACL changes — can be configured to require multi-party approval via a quorum governance system. A quorum request specifies the operation, the required number of approvals, and the eligible approvers. Each approval includes a cryptographic signature. The system supports three quorum policies: unanimous, majority, and threshold. Approved operations are executed atomically; expired requests are automatically cleaned up. A "rubber stamp" mode allows low-risk operations to bypass quorum for operational efficiency.

H. Scenario Walkthrough: Journalist Source Protection

To illustrate the system's end-to-end operation, consider a journalist who receives leaked government documents from a confidential source.

Upload and vault creation. The journalist creates a Vault Container named "Project X — Source Materials" and uploads the documents via chunked upload. Each file is encrypted with a random AES-256-GCM symmetric key. The symmetric key and the file's block Recipe are sealed inside a Vault (Algorithm 4). The Vault's Tree Seed is double-encrypted under the newsroom's custodial key. A `vault_created` entry is recorded on the ledger. The journalist receives a Verification Bundle containing the Merkle root and pristine Access Seal.

Canary configuration. The journalist configures two canary bindings on the Vault Container:

1. *Absence trigger (72 hours):* If the journalist's GitHub, Fitbit, and Slack accounts all show no activity for 72 hours, email the documents as PGP-encrypted attachments to the journalist's attorney and editor.
2. *Duress trigger:* If the journalist enters a duress code at login, immediately destroy the source identity documents and distribute the evidence package to three pre-configured news organizations via `ReleaseToPublic`.

   Because the canary bindings target the Vault Container, they automatically cover all files within it — including files added after the bindings are created.

A cascading secondary binding schedules public release 7 days after the initial absence trigger if the journalist has still not checked in.

Non-access verification. Before publication, the journalist's attorney requests a non-access proof for the entire Vault Container. The system walks every file vault within the container: all Access Seals are pristine, all ledgers contain zero `vault_read_requested` entries, and zero `key_released` entries. The attorney now has cryptographic evidence that no one — including the newsroom's IT staff — has accessed any of the source documents since upload.

Selective sharing. The journalist shares specific documents with a co-author via internal sharing (ECIES key wrapping) and with an external legal reviewer via an ephemeral share link (Tier 2). The ephemeral private key travels in the URL fragment, never touching the server.

Destruction after publication. After the story runs, the journalist destroys the source identity documents. The `DestructionEngine` (Algorithm 6) reveals the Tree Seed in a signed Destruction Proof. The journalist publishes the proof alongside the Verification Bundle. Any reader can independently verify (Algorithm 7) that the source documents were destroyed and — by checking the seal status — that they were never accessed by anyone other than the journalist.

Trigger activation. Six months later, the journalist is detained at a border crossing. After 72 hours of inactivity across all configured providers, the canary engine executes: the attorney and editor receive the evidence package. After 7 days, the cascading trigger releases the full archive publicly. The journalist's duress code was not needed because the absence trigger handled the situation automatically.

IX. Performance and Storage Analysis

A. Computational Costs

Table II summarizes the computational cost of each vault operation as a function of tree depth D.

Operation	Dominant Cost	Complexity	Time (D=10, est.)
Vault creation	Merkle tree construction + PBKDF2	O(2^D) SHA3-512 hashes + O(1) PBKDF2	~15 ms
Vault read	PBKDF2 + AES-GCM decrypt + custodial ECIES	O(1)	~5 ms
Vault destruction	ECIES sign + memory erasure	O(1)	~3 ms
Proof verification	Merkle root recomputation	O(2^D) SHA3-512 hashes	~10 ms
Non-access verification	Seal check + ledger walk	O(1) seal + O(N) ledger entries	~1 ms + O(N)
Merkle proof generation	Tree rebuild + path extraction	O(2^D) + O(D)	~12 ms
Merkle proof verification	Path recomputation	O(D) SHA3-512 hashes	<1 ms
Bloom witness query	Hash + filter lookup	O(1)	<1 ms

The dominant cost is Merkle tree construction during vault creation and proof verification, which requires 2^D - 1 SHA3-512 hash operations. At the default depth of 10, this is 2,047 hashes — approximately 10-15 ms on modern hardware. At depth 16 (65,536 leaves), creation time increases to ~500 ms with ~8 MB memory usage. The PBKDF2 key derivation (100,000 iterations by default) adds ~50-100 ms but is a one-time cost per operation.

B. Storage Overhead

Each Vault adds the following storage overhead beyond the encrypted file content:

Component	Size
Encrypted payload (AES-GCM)	File key (32 bytes) + serialized Recipe + 28 bytes overhead
AES-GCM IV + auth tag	12 + 16 = 28 bytes
PBKDF2 salt	32 bytes
Encrypted Tree Seed (ECIES)	~113 bytes (33-byte ephemeral key + 32-byte ciphertext + 16-byte tag + overhead)
Custodial public key	33 bytes
Creation ledger entry hash	64 bytes
Merkle root	64 bytes
Access seal	64 bytes
Total vault overhead	**~420 bytes + Recipe size**

The Verification Bundle adds ~170 bytes of fixed fields plus the Bloom witness. At depth 10 with a 0.001 false positive rate over 2,047 nodes, the Bloom filter is approximately 3.6 KB. The total bundle size is approximately 3.8 KB.

At the BrightChain storage layer, the TUPLE whitening model imposes a 3× block multiplier (data block + 2 randomizer blocks), and full TUPLE compliance for metadata adds approximately 5× total overhead. For a 1 MB file stored as Medium blocks (64 KB each), the storage cost is approximately 5 MB. This overhead is a deliberate tradeoff: storage density has improved exponentially while legal liability for hosting content has only increased. The storage-density advantage — measured in cheap bytes rather than expensive watts — compares favorably to proof-of-work systems.

C. Serialization Sizes

Format	Fixed overhead	Variable	CRC
VerificationBundle	170 bytes	Bloom witness (~3.6 KB at D=10) + optional proof	2 bytes
DestructionProof	106 bytes	Signature (~64-72 bytes)	—
Recipe	14 bytes	blockIdCount × blockIdSize	2 bytes
ACL Document	3 bytes	entries × ~20-50 bytes each	2 bytes

X. Security Analysis

A. Threat Model

We consider an adversary with the following capabilities:

1. **Storage access:** The adversary can read and write arbitrary bytes in the Vault's storage medium (e.g., a compromised server, a seized hard drive, or a malicious cloud provider).
2. **Network observation:** The adversary can observe all network traffic to and from the system.
3. **Computational resources:** The adversary has polynomial-time computational resources (but not quantum computers — see Section XIV).
4. **Legal compulsion:** The adversary can compel the platform operator to produce data or keys via legal process.

We assume the adversary does *not* have:

- The creator's secp256k1 private key
- The Custodian's private key (or sufficient admin quorum signatures to authorize a key release)
- The ability to modify the append-only ledger without detection (i.e., the ledger's hash chain integrity holds)

B. Provable Destruction Security

Claim 1. A valid Destruction Proof for Vault V constitutes a publicly verifiable proof that the encryption key and Recipe for V's protected file are irrecoverable.

Argument. The Destruction Proof reveals the Tree Seed, which is the sole secret from which the AES key (via PBKDF2) and thus the Secret Payload are derived. The proof includes a secp256k1 signature over the Tree Seed, a nonce, and a timestamp, binding the destruction to a specific identity and time. A verifier recomputes the Merkle tree from the revealed seed and confirms that the root matches the published commitment. Since the AES key was derived from the Tree Seed and the Tree Seed is now public, the encrypted payload can no longer protect the Secret Payload — but the Secret Payload itself (the file's encryption key and Recipe) was encrypted under this AES key, and the MemoryEraser has overwritten the plaintext. The file's encryption key is gone, and without it, the encrypted blocks in the Owner-Free Filesystem are indistinguishable from random data.

Claim 2. A Destruction Proof for Vault A cannot be replayed against Vault B.

Argument. The Merkle root derived from Vault A's Tree Seed will not match Vault B's published Merkle root (assuming SHA3-512 collision resistance). The verifier checks this match as part of the verification protocol.

C. Non-Access Security

Claim 3. If the Access Seal is pristine and the ledger contains no `vault_read_requested` or `key_released` entries for a Vault, then the Vault's Secret Payload has never been decrypted.

Argument. Decrypting the payload requires the Tree Seed. Obtaining the Tree Seed requires a custodial key release, which is recorded on the ledger before the seed is returned. The read operation records a `vault_read_requested` entry before requesting the key release. Both entries are append-only and hash-chained. The seal mutation (pristine → accessed) occurs after successful decryption. Therefore: no ledger entries implies no key release implies no Tree Seed implies no decryption. The pristine seal provides a fast local confirmation consistent with the ledger state.

Claim 4. A snapshot-restore attack (reading the Vault, then restoring the bytes to their pre-read state) is detectable.

Argument. The read operation records a `vault_read_requested` entry on the ledger *before* the seal is mutated. Restoring the Vault bytes restores the pristine seal, but the ledger entry persists. The `LedgerVerifier` detects the inconsistency: pristine seal + ledger read entries = `SealLedgerInconsistencyError`.

D. Seal Forgery Resistance

Claim 5. An adversary cannot forge a pristine seal without the Tree Seed.

Argument. Forging the seal requires finding a 32-byte value X such that HMAC-SHA3-512(X, "burn-bag-v1-pristine") equals the stored seal. This is a preimage attack on HMAC-SHA3-512, which has 2^256 security under the assumption that SHA3-512 is a pseudorandom function. The Tree Seed search space of 2^256 makes exhaustive search computationally infeasible.

E. Custodial Bypass Resistance

Claim 6. An adversary with raw storage access but without the Custodian's private key cannot decrypt the Tree Seed.

Argument. The Tree Seed is encrypted under the Custodian's ECIES public key (secp256k1). Decrypting requires the Custodian's private key or solving the Elliptic Curve Discrete Logarithm Problem (ECDLP), which is computationally infeasible for secp256k1 with current technology. The `LedgerQuorumCustodian` additionally requires admin quorum signatures for key release, preventing unilateral access even by the Custodian operator.

F. Legal Compulsion Resistance

The system is architecturally designed so that the platform operator *cannot* comply with a subpoena for Vault contents:

1. The operator does not hold the creator's private key (required for Vault read).
2. The operator does not hold the Custodian's private key (or if the operator runs the Custodian, the quorum requirement prevents unilateral release).
3. If a canary trigger has executed, the Vault is destroyed and the destruction proof is the only remaining artifact — it proves the data is gone, not what the data contained.
4. The underlying BrightChain storage provides plausible deniability at the block level — individual blocks are indistinguishable from random data due to XOR whitening.

XI. Correctness Properties and Testing

The security claims in Section X are theoretical arguments about the system's behavior under adversarial conditions. This section presents the empirical evidence that validates those claims through 511 passing tests across 35 test suites.

Each security claim maps to one or more machine-verifiable correctness properties, and each property is implemented as a test that exercises the real production code:

Security Claim	Validated By	Test Evidence
Claim 1 (Provable destruction)	Properties 7, 8, 27; Integration: full lifecycle	Destruction proof round-trips through real ECIES signing and Merkle root recomputation; tampered proofs are rejected field-by-field
Claim 2 (Replay rejection)	Property 23; Integration: replay rejection	Proof for vault A fails verification against vault B's bundle due to Merkle root mismatch, using real SHA3-512 tree construction
Claim 3 (Non-access security)	Properties 19, 15; Integration: full lifecycle	Pristine seal + zero ledger entries = non-access confirmed; reading flips both seal and ledger counters so non-access fails afterward
Claim 4 (Snapshot-restore detection)	Property 21; Integration: snapshot-restore	Seal bytes manually reverted after read; LedgerVerifier detects inconsistency because ledger entries persist
Claim 5 (Seal forgery resistance)	Property 22; Integration: seal forgery	Random seeds do not match stored seal; pristine and accessed seals are cryptographically independent
Claim 6 (Custodial bypass resistance)	Properties 17, 18, 26	Custodian refusal preserves vault state and seal; decryption without custodian key fails

The integration tests are particularly significant because they use zero mocked cryptographic components — real ECIES key generation, real AES-256-GCM encryption, real PBKDF2 key derivation, real SHA3-512 Merkle trees, and real HMAC-SHA3-512 seal derivation. The only thin wrappers are an in-memory append-only ledger (which uses real payloads and append-only semantics) and an in-memory custodian (which uses real ECIES encrypt/decrypt). This means the tests validate not just the protocol logic but the actual cryptographic operations that would execute in production.

A. Property-Based Testing Methodology

The Digital Burnbag library employs a three-tier testing strategy: unit tests for specific examples and edge cases, property-based tests for universal correctness guarantees, and end-to-end integration tests that exercise the full vault lifecycle using live cryptographic implementations with zero mocked components.

Property-based tests use the `fast-check` library to generate random inputs and verify that properties hold across all valid executions. Each property maps to one or more algorithms from Section VI.

Integration tests wire together real `ECIESService`, `AESGCMService`, `Pbkdf2Service`, `MerkleCommitmentTree`, `AccessSeal`, `DestructionEngine`, `ProofVerifier`, and `LedgerVerifier` instances — the only thin wrappers are an in-memory append-only ledger (which still uses real payloads and append-only semantics) and an in-memory custodian (which uses real ECIES encrypt/decrypt). These tests walk the full lifecycle and verify counters, seal state, and proof validity at every step:

- **Full lifecycle:** Create vault → verify pristine seal → read → verify seal mutated → verify ledger counters incremented → second read → destroy → verify destruction proof → verify seal status in proof → verify destroyed vault rejects operations.
- **Never-read provability:** Create → destroy without reading → verify proof shows pristine seal status.
- **Snapshot-restore detection:** Read vault → manually revert seal bytes → LedgerVerifier detects inconsistency.
- **Replay rejection:** Destroy vault A → verify proof fails against vault B's bundle.
- **Field-level tamper evidence:** Independently tamper tree seed, nonce, signature, timestamp, creator public key →

each causes verification failure.

- **Merkle proof selective disclosure:** Verify proofs for boundary and middle leaf indices, with tamper detection.
- **Seal forgery resistance:** Random seeds do not match stored seal.
- **Ledger entry counting:** Exact counts tracked through full lifecycle (1 after create, 3 after first read, 5 after second read, 7+ after destroy).

We define 28 machine-verifiable correctness properties organized into seven categories. For each property, we note the algorithms it validates and the source file containing the test implementation.

B. Vault Lifecycle Properties (Properties 1, 5, 6, 14)

Algorithms validated: 4 (CreateVault), 5 (ReadVault), 6 (DestroyVault). Test files: `vault.spec.ts`.

Property 1 (Payload Round-Trip). For any valid encryption key and Recipe, creating a Vault (Algorithm 4) and reading it (Algorithm 5) with the correct credentials returns the original key and an equivalent Recipe.

Property 5 (Failed Decryption Preserves Seal). A failed read attempt (wrong credentials in Algorithm 5) leaves the seal and state unchanged.

Property 6 (Destroyed Vault Rejection). For any destroyed Vault, both read (Algorithm 5) and destroy (Algorithm 6) operations are rejected with `VaultDestroyedError`.

Property 14 (State Machine Transitions). For any sequence of operations, the Vault state follows the valid transition graph: Sealed permits read (→ Accessed) and destroy (→ Destroyed); Accessed permits read (→ Accessed) and destroy (→ Destroyed); Destroyed rejects all operations.

C. Commitment and Seal Properties (Properties 2–4, 20, 22)

Algorithms validated: 1 (BuildMerkleTree), 2 (GenerateProof/VerifyProof), 3 (DeriveSeal/VerifyPristine/VerifyAccessed). Test files: `merkle-commitment-tree.spec.ts`, `access-seal.spec.ts`, `vault.spec.ts` *(Property 4).*

Property 2 (Tree Verification Round-Trip). For any random seed and valid depth, Algorithm 1 produces a root that verifies against the same seed.

Property 3 (Wrong Seed Rejection). For any two distinct seeds, computing the root from one and verifying the other fails.

Property 4 (Seal Mutation on Read). Reading a Vault (Algorithm 5) changes the seal from `DeriveSeal(seed, PRISTINE_DOMAIN)` to `DeriveSeal(seed, ACCESSED_DOMAIN)` (Algorithm 3); subsequent reads do not revert it.

Property 20 (Merkle Proof Selective Disclosure). For any leaf index, Algorithm 2's `GenerateProof` and `VerifyProof` succeed; modifying any sibling hash causes failure.

Property 22 (Seal Forgery Infeasibility). A random value that is not the Tree Seed produces a seal (Algorithm 3) that does not match the stored seal.

D. Destruction Proof Properties (Properties 7, 8, 23, 27)

Algorithms validated: 6 (DestroyVault), 7 (VerifyDestructionProof). Test files: `destruction-engine.spec.ts`, `adversarial.spec.ts`.

Property 7 (Proof Verification Round-Trip). Destroying a Vault (Algorithm 6) and verifying the proof (Algorithm 7) against the original bundle succeeds.

Property 8 (Corrupted Proof Rejection). Modifying any field of a valid proof causes Algorithm 7 to return failure.

Property 23 (Replay Rejection). A proof for Vault A fails Algorithm 7 verification against Vault B's bundle.

Property 27 (Field-Level Tamper Evidence). Independently modifying each proof field (tree seed, nonce, timestamp, signature, public key) causes Algorithm 7 to fail.

E. Serialization Properties (Properties 9–12)

Algorithms validated: 10 (CreateBloomWitness/QueryBloomWitness), 11 (SerializeBundle/DeserializeBundle). Test files: `bloom-witness.spec.ts`, `bundle-serializer.spec.ts`, `recipe-serializer.spec.ts`.

Property 9 (Bloom Witness Completeness). Every tree node is found by Algorithm 10's `QueryBloomWitness` (no false negatives).

Property 10 (Bloom Serialization Round-Trip). Serialize-deserialize preserves membership queries.

Property 11 (Bundle Serialization Round-Trip). Algorithm 11's serialize-deserialize preserves all fields; byte corruption causes CRC failure.

Property 12 (Recipe Serialization Round-Trip). Serialize-deserialize preserves all fields; byte corruption causes CRC failure.

F. Ledger and Custodial Properties (Properties 15–19, 21, 26)

Algorithms validated: 4 (CreateVault, ledger step), 5 (ReadVault, ledger step), 6 (DestroyVault, ledger step), 8 (VerifyNonAccess). Test files: `ledger-gateway.spec.ts`, `ledger-verifier.spec.ts`, `custodian.spec.ts`.

Property 15 (Ledger Records All Operations). Every lifecycle operation (Algorithms 4, 5, 6) produces a corresponding ledger entry.

Property 16 (Ledger Failure Aborts Operation). If the ledger write fails in Algorithms 4, 5, or 6, the operation is rejected and the Vault state is unchanged.

Property 17 (Custodial Key Release Round-Trip). Encrypting (Algorithm 4, step 6) and releasing the Tree Seed with valid quorum returns the original seed.

Property 18 (Custodial Refusal Preserves State). If the Custodian refuses in Algorithm 5, the seal and state are unchanged.

Property 19 (Non-Access Verification). Algorithm 8 confirms non-access when seal is pristine and ledger has zero read entries; raises inconsistency when seal is pristine but ledger has read entries.

Property 21 (Snapshot-Restore Detection). Reverting the seal to pristine after a read is detected by Algorithm 8's ledger cross-check.

Property 26 (Custodial Bypass Rejection). Decrypting without the Custodian's key fails; forging key-release entries without quorum is rejected.

G. Adversarial Properties (Properties 24, 25, 28)

Algorithms validated: 1 (BuildMerkleTree), 5 (ReadVault). Test files: `vault.spec.ts` (Property 24), `adversarial.spec.ts` (Properties 25, 28).

Property 24 (Concurrency Safety). Concurrent read operations (Algorithm 5) both record ledger entries; the Vault transitions to Accessed exactly once via the mutex.

Property 25 (Ledger Chain Integrity). Modifying any entry's payload breaks chain validation; removing an entry breaks linkage; invalid signatures are rejected.

Property 28 (Brute-Force Resistance). Two distinct seeds produce distinct Merkle roots via Algorithm 1 (collision resistance); the 2^256 search space is computationally infeasible.

XII. Implementation

The Digital Burnbag vault library (`digitalburnbag-lib`) is implemented in TypeScript and designed for deployment in both Node.js server environments and modern web browsers.

A. Codebase Structure

The library is organized into six module groups:

Module	Files	Responsibility
`crypto/`	8	Vault, VaultFactory, DestructionEngine, ProofVerifier, MerkleCommitmentTree, AccessSeal, BloomWitness, MemoryEraser
`ledger/`	3	LedgerGateway, LedgerQuorumCustodian, LedgerVerifier
`serialization/`	4	BundleSerializer, RecipeSerializer, ACLDocumentSerializer, FileMetadataSerializer
`services/`	18	FileService, FolderService, UploadService, ShareService, ACLService, CanaryService, DestructionService, QuorumService, AuditService, WatermarkService, StorageQuotaService, KeyWrappingService, NotificationService, SyncService, FolderExportService, PhixService, VaultContainerService
`interfaces/`	50+	Type definitions organized into bases, DTOs, frontend objects, service interfaces, repository interfaces, parameter types
`enumerations/`	15	VaultState, VaultContainerState, VaultLedgerEntryType, PermissionFlag, PermissionLevel, CanaryCondition, CanaryProvider, ProtocolAction, QuorumOperationType, FileAuditOperationType, etc.

B. Dependencies

The cryptographic foundation relies on audited, well-maintained libraries:

- `@noble/hashes` (SHA3-512, HMAC) — pure JavaScript, no native dependencies, audited by Cure53

- `@noble/curves` (secp256k1) — same provenance
- `@digitaldefiance/ecies-lib` — ECIES, AES-GCM, PBKDF2, CRC services built on the Noble suite
- `bloom-filters` — Bloom filter implementation

  The library uses `Uint8Array` exclusively for binary data (no Node.js `Buffer`), ensuring browser compatibility via the Web Crypto API (`crypto.getRandomValues`, `crypto.subtle`).

C. Testing Infrastructure

The test suite comprises unit tests, property-based tests, and end-to-end integration tests:

- **Property-based tests:** 28 properties (Section XI), each running 100+ iterations via `fast-check`, covering all algorithms from Section VI
- **Integration tests:** Full vault lifecycle tests using live cryptographic implementations (ECIESService, AESGCMService, Pbkdf2Service, MerkleCommitmentTree, AccessSeal, DestructionEngine, ProofVerifier, LedgerVerifier) with zero mocked cryptographic components, validating provable destruction, non-access verification, snapshot-restore detection, replay rejection, field-level tamper evidence, and ledger entry counting through every lifecycle permutation
- **Unit tests:** Edge cases (tree depth exactly 8, empty recipes, zero-length buffers), error conditions (all error types), service-level tests (VaultContainerService, FileService, FolderService, CanaryService, DestructionService, QuorumService, ACLService, ShareService, UploadService), and adversarial scenarios (snapshot-restore simulation, seal forgery, proof replay, concurrent races)
- **Test counts:** 511 tests across 35 test suites, all passing
- **Test organization:** One spec file per component, with custom `fast-check` arbitraries for domain-specific random generation (`arbTreeSeed`, `arbRecipe`, `arbLeafIndex`, `arbProofFieldMutation`, `arbLedgerTamperAction`, etc.)

D. Architecture Pattern

The library uses a repository pattern to decouple business logic from persistence. All service classes accept repository interfaces (e.g., `IFileRepository`, `ICanaryRepository`, `IShareRepository`) via constructor injection. The API layer (`digitalburnbag-api-lib`, separate package) provides concrete implementations backed by BrightDB. This separation enables:

- The same service logic to run in Node.js (with BrightDB) and in the browser (with IndexedDB or in-memory stores)
- Unit testing with mock repositories without database dependencies
- Future migration to alternative storage backends without modifying service code

---

XIII. Ethical Considerations and Dual-Use Analysis

A. Dual-Use Acknowledgment

Any system that provides provable destruction and resistance to legal compulsion is inherently dual-use. The same capabilities that protect a journalist's sources can shield criminal activity from law enforcement. We acknowledge this tension directly rather than eliding it.

B. Design Choices That Mitigate Abuse

Several architectural decisions constrain the system's potential for abuse while preserving its protective capabilities:

Quorum governance. Sensitive operations (destruction, external sharing) can require multi-party approval. An organization deploying Digital Burnbag can mandate that no single individual can destroy evidence unilaterally — destruction requires co-signatures from a configurable quorum of administrators. This provides an organizational check on abuse while preserving the cryptographic guarantees.

Immutable audit trail. Every operation is recorded on the append-only ledger. While the *contents* of a Vault are protected, the *fact* that a Vault existed, was accessed, or was destroyed is permanently recorded. This audit trail is available to authorized parties (e.g., compliance officers, courts with jurisdiction over the ledger operators) and cannot be erased.

Non-access verification cuts both ways. The same mechanism that proves a journalist's sources were not accessed also proves that a compliance officer's sealed records were not tampered with. Non-access verification is as useful for regulatory compliance as it is for source protection.

Canary protocols require pre-configuration. The dead man's switch and duress mechanisms must be configured *before* they are needed. They cannot be retroactively applied to destroy evidence after a legal hold is issued — the canary binding creation is itself recorded on the audit trail with a timestamp.

C. Legal Landscape

The legal status of cryptographic erasure varies by jurisdiction. In the United States, the Stored Communications Act (18 U.S.C. § 2701) and the Computer Fraud and Abuse Act (18 U.S.C. § 1030) govern access to stored electronic communications, but do not specifically address cryptographic erasure of one's own data. Obstruction of justice statutes (18 U.S.C. § 1519) prohibit destruction of documents "with the intent to impede" a federal investigation, but apply to the *actor's intent*, not to the *tool's capability*. A hammer can be used to build a house or break a window; the law regulates the act, not the instrument.

In the European Union, the General Data Protection Regulation (GDPR) Article 17 establishes a "right to erasure" that Digital Burnbag's provable destruction directly supports — providing cryptographic proof of compliance with erasure requests.

D. Responsible Deployment

We recommend that organizations deploying Digital Burnbag:

1. Enable quorum governance for destruction operations in corporate environments
2. Configure audit log retention policies that comply with applicable legal hold requirements
3. Use the non-access verification capability for regulatory compliance (e.g., proving that sealed grand jury materials were not accessed)
4. Provide training on the distinction between legitimate protective use and obstruction

---

XIV. Limitations and Future Work

JavaScript memory erasure. The `MemoryEraser` applies best-effort overwrite (random bytes then zeros), but JavaScript's garbage collector may copy buffers before they are wiped. WebAssembly-based secure memory or hardware-backed key storage (WebAuthn) could provide stronger guarantees.

Quantum vulnerability. The secp256k1 ECDLP and ECIES encryption are vulnerable to quantum computers running Shor's algorithm. Migration to post-quantum key encapsulation mechanisms (e.g., ML-KEM/Kyber) is planned for a future version. The SHA3-512 hash functions and HMAC constructions remain secure against known quantum attacks (Grover's algorithm provides only a quadratic speedup, reducing 512-bit security to 256-bit).

Ledger scalability. The current ledger implementation is a single-node append-only chain. For production deployment at scale, integration with a distributed consensus mechanism (e.g., BrightChain's gossip-based block propagation) is required.

Provider availability. Canary protocol reliability depends on the availability and API stability of third-party providers. Provider API changes, rate limiting, or service discontinuation could cause false absence triggers. The multi-provider redundancy model mitigates this risk but does not eliminate it.

Formal verification. The 28 correctness properties are validated through property-based testing with 100+ iterations per property, but formal machine-checked proofs (e.g., in Coq or Lean) would provide stronger guarantees. The deterministic construction of the Merkle tree and seal derivation are amenable to formal verification.

XV. Conclusion

Digital Burnbag introduces two cryptographic capabilities absent from existing secure storage and whistleblower protection systems: provable destruction and proof of non-access. The three-layer vault protection model — combining Merkle commitment seals, blockchain ledger audit, and custodial ECIES double-encryption — ensures that any access to stored secrets leaves an indelible, externally verifiable trace, while destruction produces a publicly verifiable proof that requires no trust in the platform operator.

The canary protocol engine transforms these cryptographic primitives into a practical tool for the people who need them most: journalists protecting sources, whistleblowers ensuring evidence survives their incapacitation, activists operating under surveillance, and anyone who needs mathematical certainty that their data will be destroyed on schedule or released to the right people at the right time. The system monitors activity across 40+ third-party services, supports cascading triggers with configurable delays, and detects duress conditions — providing automated protection that functions precisely when the user cannot act on their own behalf.

Built on BrightChain's Owner-Free Filesystem, where every block is XOR-whitened into plausible deniability, Digital Burnbag provides defense in depth from the storage layer to the application layer. The Vault Container architecture organizes files into independently governed collections with container-level access control, canary bindings, and aggregate non-access verification, while the seal-break confirmation protocol ensures users make informed decisions before irreversibly mutating a file's cryptographic seal.

The 28 machine-verifiable correctness properties, validated through property-based testing and end-to-end integration tests using live cryptographic implementations, provide empirical evidence that the system's security claims hold in practice. The integration tests exercise the full vault lifecycle — creation, seal verification, reading with seal mutation, ledger counter tracking, destruction with proof generation, proof verification, snapshot-restore attack detection, replay rejection, and field-level tamper evidence — using real ECIES, AES-256-GCM, PBKDF2, SHA3-512, and HMAC-SHA3-512 operations with zero mocked cryptographic components. Every security claim in Section X is mapped to specific test evidence in Section XI, bridging the gap between theoretical argument and empirical validation.

Mathematics does not accept a search warrant. Digital Burnbag makes that principle operational.

References

[1] A. Ramokapane, A. Rashid, and J. Such, "'I feel stupid I can't delete...': A Study of Users' Cloud Deletion Practices and Coping Strategies," in *Proc. USENIX Symposium on Usable Privacy and Security (SOUPS)*, 2017.

[2] M. Marlinspike and T. Perrin, "The Double Ratchet Algorithm," Signal Foundation, Tech. Rep., 2016.

[3] Freedom of the Press Foundation, "SecureDrop: The Open-Source Whistleblower Submission System," https://securedrop.org, 2013.

[4] J. Reardon, D. Basin, and S. Capkun, "Secure Data Deletion from Persistent Media," in *Proc. IEEE Symposium on Security and Privacy*, 2013.

[5] R. Kissel, A. Regenscheid, M. Scholl, and K. Stine, "Guidelines for Media Sanitization," NIST Special Publication 800-88 Rev. 1, 2014.

[6] R. Geambasu, T. Kohno, A. Levy, and H. Levy, "Vanish: Increasing Data Privacy with Self-Destructing Data," in *Proc. USENIX Security Symposium*, 2009.

[7] S. Wolchok, O. Hofmann, N. Heninger, E. Felten, J. Halderman, C. Rossbach, B. Waters, and E. Witchel, "Defeating Vanish with Low-Cost Sybil Attacks Against Large DHTs," in *Proc. NDSS Symposium*, 2010.

[8] R. Perlman, "The Ephemerizer: Making Data Disappear," *Journal of Information System Security*, vol. 1, no. 1, pp. 51-68, 2005.

[9] Hermes Center for Transparency and Digital Human Rights, "GlobaLeaks: Open-Source Whistleblowing Framework," https://www.globaleaks.org, 2012.

[10] WikiLeaks, "About WikiLeaks," https://wikileaks.org/About.html, 2006.

[11] EFF, Freedom of the Press Foundation, NYU Law, Calyx Institute, and Berkman Center, "Canary Watch," Electronic Frontier Foundation, 2015.

[12] R. Merkle, "A Digital Signature Based on a Conventional Encryption Function," in *Proc. CRYPTO*, 1987.

[13] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008.

[14] B. Laurie, A. Langley, and E. Kasper, "Certificate Transparency," RFC 6962, 2013.

[15] C. Papamanthou, R. Tamassia, and N. Triandopoulos, "Authenticated Hash Tables Based on Cryptographic Accumulators," *Algorithmica*, vol. 74, no. 2, 2016.

[16] The Owner-Free Filesystem Project, "OFF System," https://offsystem.sourceforge.net, 2006.

[17] J. Mulein, "BrightChain: A Unified Cryptographic Platform for Privacy-Preserving Decentralized Applications," Digital Defiance, Tech. Rep., 2025.